

Conway's Game Of Life in RISCv Assembly

Cache Data Report

Mohamed Azouz — 900225230
Omar Abdel Motalb — 900232118
Baraa Ben Mansour — 900244128

December 2025

1 Introduction

After developing Conway’s Game of Life using RISC-V assembly, we conducted a short study to examine the hit rates for our implementation of the game. We examined 4 different cache configurations with varying block sizes and number of blocks. The cache configurations examined are:

1. Fully associative cache with random replacement
2. Fully associative cache with LRU replacement
3. 4-way set associative cache with random replacement
4. 4-way set associative cache with LRU replacement

All 4 cache configurations were explored for block sizes 1, 2, 4, 8, and 16. They were also examined with the number of blocks set to 1, 16, 128, and 256.

2 Fully Associative Cache with Random Replacement

The figure shows a correlation between the block size and the hit rate, that is due to spatial locality where fetching larger blocks effectively pre-loads neighboring data that a program is likely to access next. The 1-block configuration performs poorly because it suffers from severe capacity misses as with only one slot, it must constantly evict and replace the data despite the random policy. Furthermore, the dramatic increase in the hit rate for all the other block configurations demonstrates that when the cache is large enough to handle the necessary data, the hit rate saturates to 99%. There is no benefit in increasing the block size from 128 to 256 as it yields no benefits because the cache is already large enough to hold the essential data required to run the game of life. Overall, the random replacement proves to be an effective policy due to evictions becoming less frequent which makes the block replacement less critical.

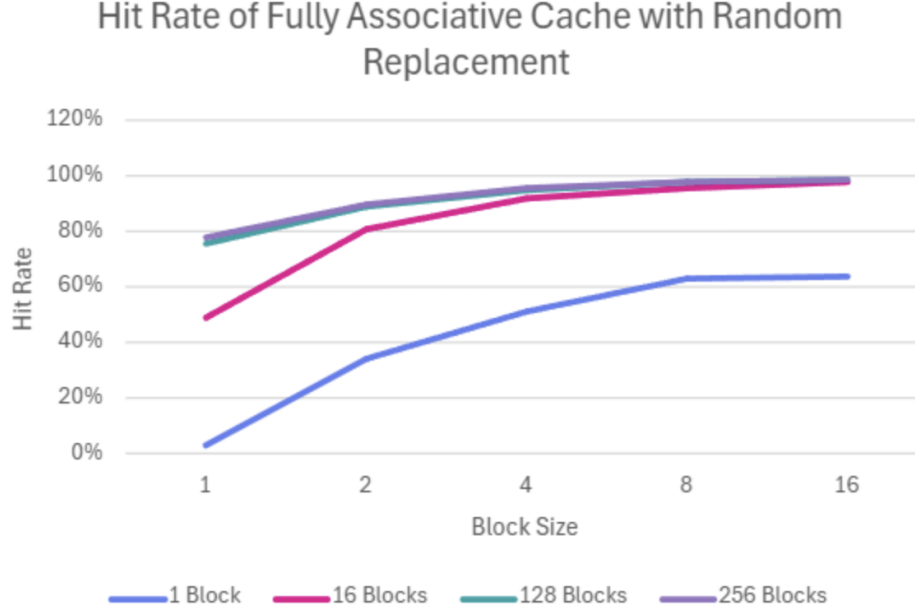


Figure 1: Graph Showing the Hit Rate against Block Size for a varying number of blocks.

3 Fully Associative Cache with LRU Replacement

The results shown in the figure benefit from the spatial locality, where increasing the block size allows the cache to pre-fetch sequential data which boosts the hit rate across all configurations. The 1-block cache performs poorly due to the continuous capacity misses and the inability of the LRU to preserve useful data due to the size. Conversely, the 16, 128, and 256-block configurations show dramatic improvements because they are large enough to utilize temporal locality by keeping recently accessed data in memory. The plateau seen from the lines indicates that the cache is already large enough to hold frequent data, so adding more blocks is no longer of use. Ultimately, while LRU is a smarter algorithm than Random, the results are nearly identical due to having sufficient cache capacity which overarches the specific replacement policy used.

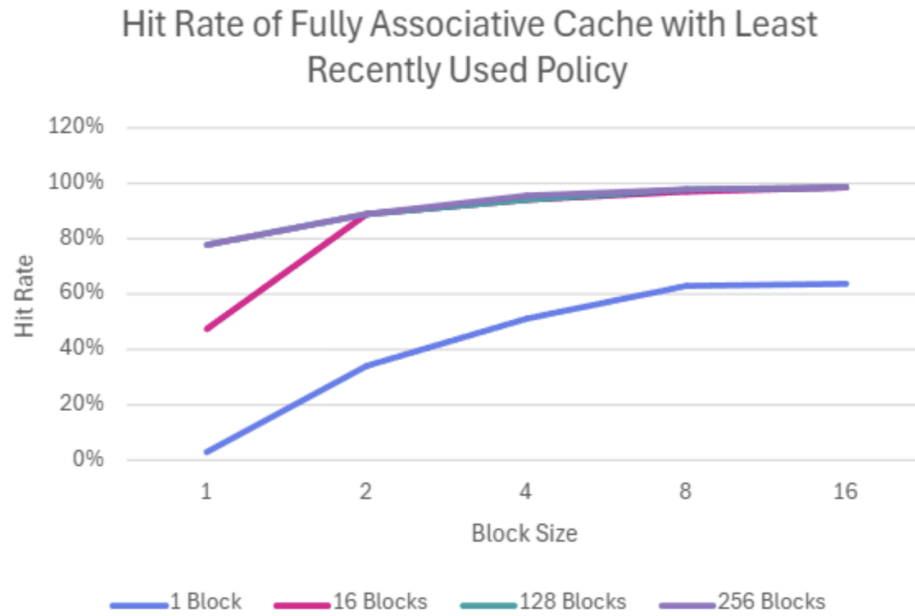


Figure 2: Graph Showing the Hit Rate against Block Size for a varying number of blocks.

4 4-way Set Associative Cache with Random Replacement

The data observed for the 4-way set associative cache configuration with random replacement followed an expected pattern.

Number of Blocks				
Block Size	1	16	128	256
1	3%	49%	71%	72%
2	34%	76%	83%	87%
4	52%	87%	92%	95%
8	63%	92%	97%	97%
16	64%	94%	99%	99%

Figure 3: Table Demonstrating Cache Hit Rate for varying Block Sizes and Number of Blocks.

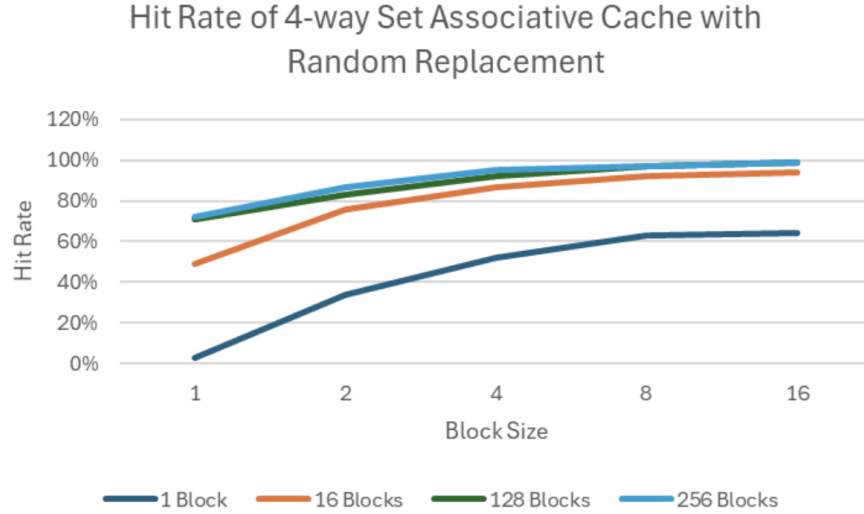


Figure 4: Graph Showing the Hit Rate against Block Size for a varying number of blocks.

It can be clearly concluded that the block size increases with the number of blocks and the block size. The increase with respect to the block size is expected. This is due to the way the memory is accessed. Since the program is continuously accessing the current and next state arrays in the memory sequentially, the spatial locality advantage of larger block sizes is used to its full extent. For each block size of n , it is guaranteed that after the first miss, the next $n-1$ memory accesses are a hit. This is why we see a dramatic increase from 3% to 64% from block size 1 to 16 for 1 block. Increasing the block sizes for the same number of blocks also increases the cache size, as the cache size is the product of the block size and the number of blocks. This makes the cache store much more data, leading to a higher probability of a hit despite the increased cache access time.

For 1 block, we can see that the hit rate is much lower than the other configurations. This is because with 1 block, it is not technically a 4-way set associative cache, but a direct-mapped cache with 1 block. This makes every new block evict the old one, which is inefficient. We also never hit 100% in our results, even when the cache is very large. This is expected due to cold start misses when the program first starts executing. We also must note that there are no decimals in the hit rates due to the nature of the data provided by the simulator.

5 4-way Set Associative Cache with LRU Replacement

The following table shows a clear trend where the hit rate improves as both the block size and the number of blocks increase. For a fixed block size, adding more blocks significantly raises the hit rate due to greater cache capacity, reducing capacity misses. For instance, at a block size of 4 bytes, the hit rate jumps from 51% with 1 block to 86%, 95%, and 96% as the number of blocks increases to 16, 128, and 256, respectively. Similarly, for a fixed number of blocks, increasing the block size improves performance by exploiting spatial locality. The benefits are most dramatic for smaller caches; with only 1 block, hit rate rises from 2% to 64% as block size grows from 1 to 16 bytes.

	Number of Blocks			
Block Size	1	16	128	256
1	2	47	76	77
2	33	74	87	91
4	51	86	95	96
8	62	91	98	98
16	64	92	99	99

Figure 5: Results Table Showing Hit Rate for different block sizes and number of blocks.

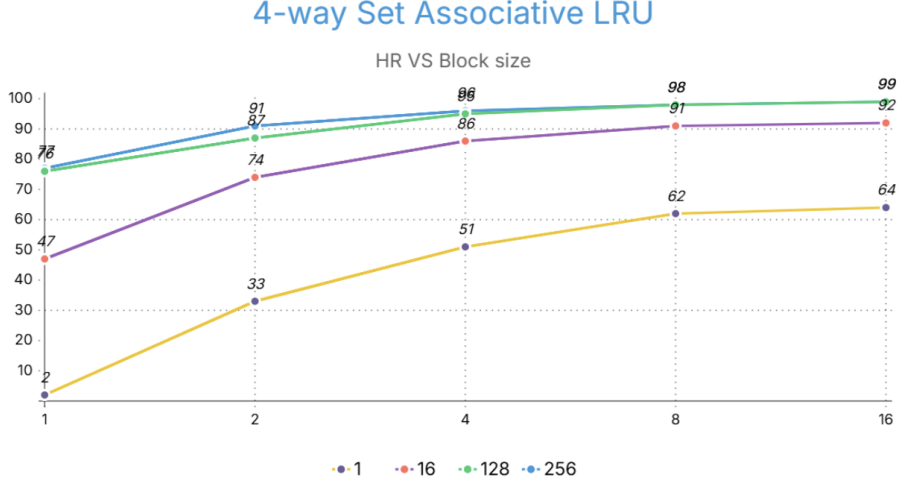


Figure 6: Graph Showing the Hit Rate against Block Size for a varying number of blocks.

The graph plots hit rate against block size for different cache sizes (1, 16, 128, and 256 blocks) in a 4-way set associative LRU cache. Each curve represents a fixed number of blocks and shows a strong positive relationship between block size and hit rate, especially for smaller caches. The curve for 1 block starts very low but climbs steeply, highlighting how even a tiny cache can achieve moderate hit rates by using larger blocks to capture spatial locality. The curves for larger caches (128 and 256 blocks) start much higher and approach near-perfect hit rates (98–99%) quickly, plateauing after a block size of 8 bytes. This illustrates that while increasing block size helps all configurations, its impact is most pronounced when the cache is very small. Once the cache is sufficiently large (e.g., 128 or 256 blocks), hit rates become very high and further gains from larger blocks are minimal.

6 Conclusion

This report analyzed the cache performance of our RISC-V implementation of Conway’s Game of Life under four different cache configurations: fully associative and 4-way set associative, each with random and LRU replacement policies. Across all configurations, we observed consistent trends: hit rates improve significantly with both increased block size and number of blocks, reflecting the benefits of spatial and capacity locality. Larger block sizes effectively exploit the sequential memory access patterns in our program, while more blocks reduce capacity and conflict misses. These results demonstrate that for our memory-access-intensive Game of Life simulation, a balanced cache design can achieve high efficiency even within constrained hardware environments.